

过去 24 小时合并的 PR 深度解读

报告日期：2026-06-23 | 仓库：[sgl-project/sglang-omni](#) | 抓取窗口：
merged ≥ 2026-06-22

5

合并 PR 总数

3

深度解读 (Omni / 多模态 / 多阶段)

2

CI / 基础设施 (简述)

+48%

本日最大单项吞吐提升 (#822)

多模态 TTS 声码器 (Vocoder)

SGLang 变长 FlashAttention

多阶段 Pipeline 架构

CUDA Graph / torch.compile 批策略

KV Cache 显存预算

CI 阈值校准

CI 工作流编排

今日总览 · 主题分类

今天的 5 个 PR 可以分成两组。第一组 (#822 / #843 / #864) 是真正"碰到模型与推理引擎内核"的改动——它们围绕 SGLang-Omni 的多模态语音合成 (TTS) 流水线展开：怎么把声码器算得更快、怎么保证一堆 TTS 模型的批处理配置都"自洽"、怎么给显存预算留够余量。这三个是本报告的重点，会逐行讲代码、把每个陌生概念用"它是什么 → 为什么需要 → 本 PR 怎么用"三段式讲透。第二组 (#863 / #857) 是工程基础设施——CI 性能阈值的日常校准，以及 CI 工作流编排的 Bug 修复，和模型架构关系不大，因此放在后面简述，但同样点出它们背后的工程思想。

#822 为 MOSS 声码器接上 SGLang 变长 FlashAttention 解码路径 深度 · 多模态/多阶段

#843 校验所有 TTS 生成阶段的批处理策略 (CUDA Graph / torch.compile) 深度 · 多阶段

#864 抬高 colocated 声码器显存预留，给 CI 留出 VRAM 余量 深度 · 显存/KV

#863 2026/06/22 CI 阈值校准 简述 · CI

#857 修复定向重跑最新 Omni CI 运行的逻辑 简述 · CI

深度解读

#822 [MOSS] Add SGLang-backed MOSS vocoder decoder

MERGED

多模态/多阶段

作者 Ratish1 · 合并于 2026-06-22 19:47 UTC · [查看 PR ↗](#)

1. PR 概览

一句话：给 MOSS-TTS Local 的"声码器"在非流式解码时换上一条更快的注意力计算路径——把原本由上游库执行的 transformer 注意力，改走 SGLang 打包后的变长 **FlashAttention** (packed varlen FlashAttention)，在 H100 上把这一段的吞吐拉高约 **+48.6%**、平均时延降低约 **-33%**，而音质指标 (WER / UTMOS / 说话人相似度) 基本持平。

2. 它解决了什么问题

要听懂这个 PR，先要知道 TTS (文本转语音) 系统通常分两大步，这正是"多阶段架构"的含义：

1. AR 阶段 (自回归语言模型)：像 LLM 一样，一个 token 一个 token 地生成"音频 token" (codec codes，可以理解成压缩后的声音编码)。
2. 声码器阶段 (Vocoder)：把这些离散的音频 token 还原成真正能播放的连续波形 (waveform)。

痛点在于：声码器本身也包含一个不小的 transformer，而它在端到端时延里占了很大一块。原来的非流式路径直接调用上游

`processor.decode_audio_codes`，里面的注意力用的是通用实现 (SDPA)，并且为了对齐 padding 还要在一个 `[batch, max_seq_len, dim]` 的稠密张量上算注意力——其中有大量 padding 的"空位"也被白白算了一遍。批次里句子长短不一时，这种浪费尤其明显。

于是作者想：能不能把声码器 transformer 这段，也接到 SGLang 已经高度优化的变长 **FlashAttention** 内核上？把一个 batch 里所有句子的有效帧"压扁"成一根长序列，只算真实 token、不算 padding，从而又快又省显存。

3 · 具体做了什么改动

核心文件

- `sglang_omni/models/moss_tts_local/vocoder_decoder.py` (新增, 约 480 行) —— 本 PR 的主体, 定义了一整套“包装器”类, 把上游声码器的注意力换成打包 FlashAttention。
- `sglang_omni/models/moss_tts_local/streaming_vocoder.py` (修改) —— 在调度器里安装/卸载这个新解码器。

改动前 vs 改动后 (执行路径对比)

PR 描述里给出的这张“流程图”非常关键, 它说明新路径只替换了中间一段, 前后的 `codec` 嵌入、波形投影都保持原样 (这叫“作用域内替换 / scoped replacement”) :

非流式 MOSS codes

- tokenizer 整段解码 (`chunk_duration=None`)
- 【作用域内】安装 MOSS 声码器解码器替身
- 上游 codec 嵌入 / `pretransform` / 波形投影 ← 保持不变
- projected transformer 各阶段 :
 - input projection (输入投影)
 - pack valid frames (把有效帧打包压扁) ← 新增
 - MOSS 位置编码 / RoPE
 - SGLang `flash_attn_varlen_func` ← 核心替换点
 - output projection (输出投影)
 - unpack valid frames (解包还原成 batch) ← 新增
- waveform (波形)

而流式 (`streaming`) 请求继续走老的有状态路径, 新解码器只在非流式解码时用 `try/finally` 临时装上、用完即恢复——这是一个很克制、低风险的设计。

关键代码逐块讲解

下面这段是“打包”的核心——把 `[batch, max_seq_len, dim]` 的稠密张量, 按每句真实长度压成一根 `[total_valid_tokens, dim]` 的长序列 :

```
def _pack_padded_sequence(x, input_lengths):
    batch_size, max_seqlen, _ = x.shape
    positions = torch.arange(max_seqlen, device=x.device)
    # valid_mask: 每个位置是否在该句的真实长度内 (True=真实 token, False=padding)
    valid_mask = positions.view(1, max_seqlen) < input_lengths.view(batch_size, 1)
    packed_x = x[valid_mask]  # 只保留真实 token, 压缩成一根长条
    # cu_seqlens: 累积长度数组, [0, len0, len0+len1, ...], 标记每句在长序列里的起始位置
    cu_seqlens = torch.zeros(batch_size + 1, dtype=torch.int32, device=x.device)
    cu_seqlens[1:] = torch.cumsum(input_lengths.to(torch.int32), dim=0)
    position_ids = positions.view(1, max_seqlen).expand(batch_size, -1)[valid_mask]
    return packed_x, valid_mask, cu_seqlens, position_ids
```

- **valid_mask** : 一个布尔矩阵, 标出哪些格子是真 token, 哪些是 padding。逻辑是“位置下标 < 这句的真实长度”。
- **packed_x = x[valid_mask]** : 用掩码做花式索引, 一步就把所有真实 token 抽出来拼成一根长条, padding 全部丢掉。
- **cu_seqlens** (cumulative sequence lengths, 累积序列长度) : 这是变长 FlashAttention 的“灵魂参数”。它告诉内核“长条里第 0~len0 个 token 属于第一句、len0~len0+len1 属于第二句……”, 这样 FlashAttention 才知道注意力不能跨句子串味。
- **position_ids** : 每个 token 在它自己句子里的位置下标, 供 RoPE 位置编码使用。

然后是真正调用 SGLang 内核的地方 (在

MossTTSLocalAttention._forward_packed_flash 里) :

```

q, k, v = self._project_qkv(x)                # 线性投影出 Query/Key/Value
q, k = self._apply_packed_rope(q, k, position_ids, max_positions=max_s
out = self._flash_attn_varlen(
    q.contiguous(), k.contiguous(), v.contiguous(),
    cu_seqlens, cu_seqlens,                # q 和 k 用同一套边界（自注意力）
    max_seqlen, max_seqlen,
    causal=self.causal,                    # 因果掩码：每个 token 只能看自己和前面的
    window_size=self._flash_window_size(), # 局部窗口：只看最近 context
)
return self.out_proj(out.reshape(x.shape[0], self.embed_dim))

```

这里有个很精细的细节，体现作者对“语义对齐”的较真——局部窗口大小的换算：

```

def _flash_window_size(self):
    if self.context is None or not self.causal:
        return (-1, -1)                # (-1, -1) 表示不限制窗口
    # MOSS 的 SDPA 局部掩码：context 个 token 是"含当前 token"在内的总数
    # 而 FlashAttention 的左窗口参数：数的是"当前 token 之前的 key 数量"
    return (max(int(self.context) - 1, 0), 0)

```

为什么减 1？两套实现对“窗口”的定义差一个：上游 SDPA 把“能看见的总 token 数（含自己）”叫 context，而 FlashAttention 的左窗口只数“自己前面的 key 个数”。如果不减 1，新路径会比原实现多看一个 token，结果就对不上了。这种“差一”（off-by-one）正是替换内核时最容易踩、又最难查的坑。作者特意写注释标明，是非常专业的做法。

最后把长条还原回原来的 batch 形状（解包），保证后续波形投影看到的张量形状和原来一模一样：

```
def _unpack_packed_sequence(packed_x, valid_mask, batch_size, max_seqlen):
    x = packed_x.new_zeros(batch_size, max_seqlen, packed_x.shape[-1])
    x[valid_mask] = packed_x          # 把算好的真实 token 按掩码放回原位，padding 补 0
    return x
```

另外作者还做了一个快路径优化：当 batch 里只有一句、且它没有 padding（`batch_size==1` and 真实长度==max_seqlen）时，根本不需要构造掩码，直接 `reshape` 就行，省掉掩码开销——见 `_pack_unpadded_sequence`

诚实的工程态度：它没有“装作 bit-identical”

PR 反复强调：新路径的波形输出不是和上游逐比特相同（not bit-identical）。更难得的是，作者做了“消融实验”去定位漂移来源：他发现即便换成自己写的 SDPA 解码器（不用 FlashAttention），漂移也差不多大——这说明漂移主要来自“非流式执行模型的整体改变”，而不是 FlashAttention 打包/窗口有 Bug。这种“不甩锅、把可疑路径一条条排除”的验证方式，是性能优化 PR 的黄金标准。

4. 涉及的知识点（重点）

声码器 Vocoder

它是什么：把“声音的压缩编码（codec tokens / 离散单元）”还原成真正能播放的连续音频波形的模块。可以理解成音频领域的“解压缩器 + 渲染器”。

为什么需要它：语言模型擅长生成离散 token，但人耳要听的是连续波形。声码器就是这两个世界之间的桥。在 MOSS 里它本身还内嵌一个 transformer，所以并不便宜。

本 PR 怎么用：把声码器 transformer 那一段的注意力换成 SGLang 内核，但保留它的嵌入和波形投影，只优化“最贵的中间段”。

FlashAttention 与「变长 (varlen / packed)」

它是什么：FlashAttention 是一种把注意力计算"分块、不落地中间大矩阵"从而又快又省显存的内核。变长版本允许你把一批长短不一的句子拼成一根长序列来算，靠 `cu_seqlens` 这个边界数组区分句子。

为什么需要它：传统做法要把短句 padding 到最长句的长度，padding 部分白算、白占显存。变长版本"只算真 token"，batch 里长短差异越大，省得越多。

本 PR 怎么用：`_pack_padded_sequence` 负责压扁 + 生成 `cu_seqlens`，再交给 `flash_attn_varlen_func`，最后解包还原。

RoPE (旋转位置编码 Rotary Position Embedding)

它是什么：一种把"token 在序列里的位置"信息，通过对 Query/Key 向量做"旋转"来注入的技术。位置越靠后，旋转角度越大。

为什么需要它：注意力本身对顺序"无感"，必须额外告诉它谁前谁后。RoPE 的好处是相对位置关系天然体现在向量内积里。

本 PR 怎么用：打包成长序列后，每个 token 必须用"它在自己句子里的位置"来旋转（而不是在长条里的全局位置），所以 `position_ids` 要随打包一起算好；`_MossPackedRopeCache` 还把 cos/sin 表缓存起来避免重复计算。

SDPA (Scaled Dot-Product Attention)

它是什么：PyTorch 内置的"标准"缩放点积注意力实现（`torch.nn.functional.scaled_dot_product_attention`），通用、正确，但不一定最快。

为什么需要它：作为对照基线和兜底路径——本 PR 里当不满足条件（非 CUDA、非 bf16 等）时仍回退到上游 SDPA 路径。

本 PR 怎么用：`resolve_attention_implementation` 会判断能否走打包 FlashAttention，不能就回退 SDPA；作者还用 SDPA 做消融实验来证明漂移不是 FlashAttention 的锅。

作用域内替换 (Scoped / Monkey-patch Replacement) + try/finally

它是什么：临时把对象的某个组件换成自定义版本，用完立刻还原，保证不污染其他代码路径。

为什么需要它：流式和非流式共用同一个 codec 实例；如果永久替换会影响流式。临时替换 + `try/finally` 恢复，把风险锁在一个很小的窗口内。

本 PR 怎么用：只在非流式整段解码期间安装

`MossTTSLocalVocoderDecoder`，结束即恢复，流式会话完全不受影响。

一个类比帮助理解打包：想象食堂打饭，原来不管你要多少，都给你一个最大号餐盘并铺满 (padding)，盘子很多地方是空的还得端着走 (白占资源)。变长 FlashAttention 是改成“流水线传送带”：把所有人的实际饭量首尾相接放上同一条传送带 (packed)，只在每个人的饭之间放一道隔板 (`cu_seqlens`)，既不浪费盘子，也不会让你的饭和别人的混在一起 (因果 + 边界)。

5. 我能学到什么 / 延伸思考

可复用的工程思想

- 只优化最贵的一段，其余保持不变。不重写整个声码器，只替换 transformer 注意力，把改动面和风险都压到最小。
- 新旧并存。按条件路由 + 兜底回退。流式走老路，非流式走新路，不满足条件回退 SDPA——永远留一条已知正确的退路。
- 诚实地承认“非 bit-identical”，并用消融实验定位漂移来源。比起声称“完全等价”，量化漂移幅度 + 排除法定位，更可信也更专业。
- 对“差一”语义差异极度敏感 (窗口大小那段注释)，这是跨实现替换内核的核心功力。

想深入可以再补：注意力机制与 KV cache 原理 → FlashAttention 论文 (分块 softmax / 不落地中间矩阵) → 变长 attention 的 `cu_seqlens` 约定 → RoPE

的数学推导 → 音频 codec (如残差向量量化 RVQ) 与声码器 (HiFi-GAN 一类) 的基础。

#843 [TTS] Validate generation batch policies (#836 W1)

MERGED

多阶段

作者 [luojiaxuan](#) · 合并于 2026-06-22 02:26 UTC · [查看 PR ↗](#)

1. PR 概览

一句话：新增一个统一的“生成阶段批处理策略校验器”，把所有 SGLang 后端的 TTS / Omni 模型（Higgs、MOSS、MOSS-Local、Qwen3-TTS、Voxtral、FishAudio S2-Pro、Qwen3-Omni talker）的 CUDA Graph、torch.compile、最大并发等配置，在启动时强制检查一遍——配置不自洽就直接报错，而不是等到线上跑崩。

2. 它解决了什么问题

SGLang-Omni 支持七八个不同的语音模型，每个模型在 `stages.py` 里各自写一套启动参数（`overrides` 字典）。这里面有几个参数互相必须配套，但以前是各写各的、容易写错：

- `max_running_requests`：最大并发请求数（引擎同时处理多少条）。
- `cuda_graph_max_bs` / `cuda_graph_bs`：CUDA Graph 捕获的最大批大小、以及具体捕获哪些批大小（“分桶”列表）。
- `torch_compile_max_bs`：torch.compile 编译覆盖到的最大批大小。
- 模型自己分配的批缓冲区容量（如 KV cache、采样池的行数）。

痛点：如果 `cuda_graph_max_bs` 比 `max_running_requests` 小，那么当并发冲高、批大小超过 CUDA Graph 覆盖范围时，引擎要么回退到慢的 eager 模式、要么直接越界崩溃。更隐蔽的是 FishAudio S2-Pro 这种模型，`torch.compile` 只编到 batch=16，但实际并发能到 64——大批次时性能悬崖式下跌，却没人在启动时拦住它。这类“配置漂移”在生产里极难排查，因为启动一切正常，只在特定负载下出问题。

3 · 具体做了什么改动

核心文件

- `sglang_omni/scheduling/generation_batch_policy.py` (新增, 196 行) —— 校验器与默认分桶生成器。
- 七个模型的 `stages.py` 各自接入这个校验器 (Higgs / MOSS / MOSS-Local / Qwen3-TTS / Voxtral / FishAudio S2-Pro / Qwen3-Omni talker)。
- 两个模型 (Higgs、FishAudio) 还暴露了“模型侧真实缓冲区容量”的属性, 供校验器交叉核对。

关键代码 A : 默认 CUDA Graph 分桶, 对齐 SGLang 原生桶

```
def build_default_cuda_graph_bs(max_bs: int) -> list[int]:
    max_bs = int(max_bs)
    if max_bs < 1:
        raise ValueError("max_bs must be >= 1")
    values = [1, 2, 4, 8, 12]          # 小批次密集采样
    values.extend(range(16, 257, 8))   # 16~256, 每 8 一档
    values.extend(range(272, 512, 16)) # 256~512, 每 16 一档
    values.extend(range(512, max_bs + 1, 32)) # 512 以上, 每 32 一档
    values = [bs for bs in values if bs <= max_bs]
    if not values or values[-1] != max_bs:
        values.append(max_bs)          # 保证最后一桶正好等于 max_bs
    return values
```

这段的用意是：不要让“校验”反而缩小了 CUDA Graph 的覆盖范围。它生成的分桶刻意和 SGLang 默认捕获的桶一致——小批次密、大批次疏（因为小批次更常见、也更需要精细优化）。比如 `max_bs=64` 会得到

```
[1, 2, 4, 8, 12, 16, 24, 32, 40, 48, 56, 64]
```

关键代码 B：校验器的核心判断

```
# CUDA Graph 开启时：max(cuda_graph_bs) 必须正好等于 cuda_graph_max_bs
if max(cuda_graph_bs) != cuda_graph_max_bs:
    errors.append("max(cuda_graph_bs) must match cuda_graph_max_bs ...")

# CUDA Graph 的最大批必须 >= 最大并发，否则高并发时没图可用
if cuda_graph_max_bs < max_running_requests:
    errors.append("cuda_graph_max_bs must cover max_running_requests .")

# 只有当 torch.compile 真的开启时，才要求它覆盖最大并发
if torch_compile_enabled and torch_compile_max_bs < max_running_requests:
    errors.append("torch_compile_max_bs must cover max_running_requests .")

# 模型实际分配的缓冲区，必须能装下最大并发
if normalized_model_buffer_bs < max_running_requests:
    errors.append("model_buffer_bs must cover max_running_requests ...")

if errors:
    raise ValueError(f"{model_name} invalid generation batch policy: ")
```

逻辑可以归纳成一句话：“凡是声明能跑 N 路并发，那么 CUDA Graph、torch.compile、模型缓冲区这三道关卡，每一道都必须 $\geq N$ 。”任何一道没覆盖到，启动即 `ValueError` 失败（fail fast，快速失败），而不是留到线上炸。

关键代码 C：模型侧暴露真实容量，做“交叉核对”

光看配置不够——配置写得对，但模型实际分配的缓冲区可能更小。所以 Higgs 和 FishAudio 把真实容量暴露成属性：

```

# FishAudio S2-Pro: 从已分配的 KV cache 张量第 0 维直接读出真实批容量
@property
def kv_cache_max_batch_size(self) -> int:
    kv_cache = self.layers[0].attention.kv_cache
    return int(kv_cache.k_cache.shape[0])    # k_cache 形状是 [batch, ..

# 取 VQ 解码缓冲 与 KV cache 两者的较小值, 作为真实可用批上限
def _resolve_s2pro_model_buffer_bs(model):
    return min(int(model.vq_decode_max_batch_size),
               int(model._audio_decoder.kv_cache_max_batch_size))

```

这里的精髓是取 `min`：一个请求要同时占用“VQ 解码缓冲”和“KV cache”两块资源，木桶效应下，真实并发上限是两者里更小的那个。校验器拿这个真实值和 `max_running_requests` 对比，杜绝“配置吹牛、缓冲不够”。

顺带的实际收益

因为强制对齐，FishAudio S2-Pro 的 `torch.compile` 从只编到 `max_batch_size=16` 提升到 `64`（覆盖了它真实的 64 路并发）。Higgs 暴露了 `sampler_pool_max_running_requests`，Qwen3-Omni talker 也补齐了显式的 `cuda_graph_bs / max_running_requests / torch_compile_max_bs`。PR 还附了大量 benchmark 证明这些对齐没有引入性能回退（多数模型在噪声范围内持平）。

4. 涉及的知识点 (重点)

CUDA Graph

它是什么：把一连串 GPU 内核调用"录制"成一张静态图，之后用一次"重放"代替成百上千次逐个 launch。

为什么需要它：小模型 / 小批次推理时，瓶颈往往不是算力，而是 CPU 逐个发射内核的开销 (kernel launch overhead)。CUDA Graph 把这部分开销几乎抹平。代价是图按"固定形状"录制，所以要预先为**每个会用到的批大小**各录一张——这就是"分桶 (buckets)"。

本 PR 怎么用：校验 `cuda_graph_bs` 这些桶的最大值要等于 `cuda_graph_max_bs`、且要 $\geq$ 最大并发，保证任何实际批大小都有一张录好的图可用。

torch.compile

它是什么：PyTorch 2.x 的即时编译器，把动态 Python 模型代码编译成融合后的高效内核。

为什么需要它：同样为了减少开销、融合算子、提速。但它也要针对具体形状/批大小编译，超出编译覆盖的批大小会触发重新编译或回退，性能骤降。

本 PR 怎么用：只在 `enable_torch_compile=True` 时才强制要求 `torch_compile_max_bs  $\geq$  max_running_requests` (关掉时不卡)，把 S2-Pro 从 16 提到 64。

max_running_requests (最大并发 / continuous batching 的容量)

它是什么：推理引擎同一时刻最多并行处理多少条请求。在 SGLang 这类系统里，请求是动态进出的 (continuous batching，连续批处理)。

为什么需要它：它决定了 KV cache、采样池、CUDA Graph 等所有"按批分配"的资源该开多大。它是这套校验的"基准线 N"。

本 PR 怎么用：所有其他上限都必须 $\geq$ 它；模型真实缓冲区也必须能装下它。

Fail Fast (快速失败) 与不变量校验 (Invariant Validation)

它是什么：把“系统必须始终成立的前提（不变量）”在最早的时刻（启动时）显式检查，一旦违反立刻报错退出。

为什么需要它：配置类 Bug 如果拖到运行时才暴露，现场可能是奇怪的越界 / 性能悬崖，极难定位。在启动时一句清晰的 `ValueError` 能省下几小时排查。

本 PR 怎么用：整个 `validate_generation_batch_policy` 就是把这些不变量集中、显式地写出来并强制执行。

类比：这像飞机起飞前的检查单 (checklist)。“载客 N 人 $\rightarrow$ 救生衣 $\geq N$ 、氧气面罩 $\geq N$ 、安全带 $\geq N$ ”，任何一项不够，飞机根本不许起飞，而不是飞到半空才发现救生衣不够。本 PR 就是给七个 TTS 模型装上了同一份起飞检查单。

5. 我能学到什么 / 延伸思考

可复用的工程思想

- 把分散在各处、容易写错的“隐式约定”，抽成一个集中、显式、可测试的校验器。这是治理“配置漂移”的标准手法。
- 配置不能只信声明，要和“运行时真实分配的资源”交叉核对（读 KV cache 张量的真实形状、取 min）。
- “快速失败”优于“悄悄降级”：宁可启动直接报错，也不要线上留一个性能悬崖。
- 给每条规则都配单元测试（新增 134 行测试，覆盖每种违规场景），让校验器本身也可信。

想深入可以再补：CUDA Graph 捕获/重放机制 $\rightarrow$ torch.compile

(TorchDynamo/Inductor) 的形状特化 $\rightarrow$ continuous batching 与 KV cache 分页 $\rightarrow$ “契约式设计 / Design by Contract”中的前置条件 (precondition) 思想。

#864 [MOSS] Raise colocated codec_mem_reserve 0.05 → 0.15 for CI VRAM headroom

MERGED

显存/KV

作者 JiaxinD · 合并于 2026-06-23 02:57 UTC · [查看 PR ↗](#)

1 · PR 概览

一句话：把 MOSS-TTS Local "同卡共置 (colocated)" 模式下给声码器预留的显存比例 `codec_mem_reserve` 从 `0.05` 抬到 `0.15` (一行改动)，从而把 AR 引擎实际可用的 `mem_fraction_static` 从 0.85 降到 0.70，给同卡上的声码器和解码激活腾出约 12 GB 余量，避免 CI 在 96GB H20 上贴着显存上限跑、随时 OOM。

2 · 它解决了什么问题

"同卡共置 (colocated)" 是指：一个 CI worker 独占一张卡，把整条流水线 (预处理 + AR 引擎 + 声码器) 全塞进同一张 GPU 的同一个进程。这三者要共享同一块显存，必须分蛋糕：

- 总预算 `_COLOCATED_TOTAL_GPU_MEMORY_FRACTION = 0.90` (整卡的 90%)。
- AR 引擎的有效份额 = 总预算 0.90 - 声码器预留 `codec_mem_reserve`。原来预留 0.05，所以 AR 拿 0.85。

痛点：AR 引擎拿 0.85 时，它会用这份显存去开 KV cache 池。实测在 80GB H100 上，这个池被撑到约 39.5 万 token，是真实需要量 (`max_running_requests 16 × context_length 8192 = 131072` token) 的约 3 倍——严重过度预留。AR 把显存吃得太狠，留给声码器 (第二个 codec 实例) 和解码激活的空间就不够了。在更大的 96GB H20 上，这个"过度预抓"的绝对值更大，于是 CI 的两个 worker 逼近 ~97.6 GB / 96GB (约 99.6%)，离 OOM 只剩几百 MiB。

3 · 具体做了什么改动

核心就一行，在 `sglang_omni/models/moss_tts_local/config.py`

```
# colocated 同卡共置的显存契约
_COLOCATED_TOTAL_GPU_MEMORY_FRACTION = 0.90

- _COLOCATED_CODEC_MEM_RESERVE = 0.05    # 声码器只预留 5% → AR 拿 0.85 (过)
+ _COLOCATED_CODEC_MEM_RESERVE = 0.15    # 声码器预留 15% → AR 降到 0.70,
_AR_MEM_FRACTION_STATIC = 0.85
```

作者强调：`mem_fraction_static` 只决定 KV cache 池开多大，不改变任何生成数学（权重、采样、声码器都没动）。降到 0.70 后，KV 池仍约为真实需求的 2.35 倍，绰绰有余，所以请求准入 / 抢占行为不变，输出逐比特相同。这是一个“零行为风险”的容量旋钮调整。

有效份额	KV 池 token 数	相对真实需求(131072)	空载 VRAM
0.85 (改前)	395457	3.02×	77089 MiB
0.70 (改后)	308460	2.35×	64883 MiB

净省约 **12206 MiB (≈11.9 GB)**，且 n=3 重复测量方差为 0（完全确定性）。外推到 96GB H2O：峰值约从 97.6 GB 降到约 83 GB，腾出约 14 GB。

4 · 涉及的知识点（重点）

KV Cache（键值缓存）

它是什么：自回归生成时，每生成一个新 token 都要和前面所有 token 做注意力。把前面 token 算过的 Key / Value 向量缓存下来，避免每步重算，就是 KV cache。

为什么需要它：没有它，生成第 n 个 token 就要重算前 n-1 个的注意力，复杂度爆炸。有了它，每步只算增量。代价是它**极占显存**，且占用 $\propto$ 并发数 $\times$ 序列长度。

本 PR 怎么用：`mem_fraction_static` 决定 KV 池预抓多少显存。本 PR 发现它被开得过大（3×需求），收一收就省出十几 GB。

mem_fraction_static (静态显存份额)

它是什么：SGLang 的关键旋钮，指定“预先静态抓取整卡多大比例的显存来建 KV 池”。

为什么需要它：推理系统倾向于一次性预抓显存，避免运行时频繁分配/碎片化。但抓太多会挤占同卡上别的组件。

本 PR 怎么用：通过抬高 codec 预留，间接把 AR 的有效

`mem_fraction_static` 从 0.85 压到 0.70，给声码器让路。

Colocation (同卡共置)

它是什么：把流水线的多个阶段（预处理 / AR / 声码器）放在同一张 GPU、同一进程里跑，而不是分散到多卡多进程。

为什么需要它：省卡、省机间通信、CI 里一个 worker 一张卡就够。但所有阶段要抢同一块显存，必须精细分配预算。

本 PR 怎么用：正是 colocated 模式下“AR 太贪、声码器没饭吃”导致逼近 OOM，本 PR 重新切分显存蛋糕。

OOM (Out Of Memory) 与 Headroom (余量)

它是什么：OOM 即显存耗尽导致进程崩溃；headroom 指刻意留出的安全余量。

为什么需要它：显存用量会随负载、序列长度、碎片波动；贴着上限跑，任何抖动都会触发 OOM。

本 PR 怎么用：从“99.6% 占用、几百 MiB 余量”改成留出约 14 GB headroom，让 CI 稳定不挂。

类比：三个人合住一间屋（同卡），原来 AR 这个室友把客厅 85% 都堆了自己的杂物（其实他只需要不到一半），结果声码器室友连转身的地方都没有，屋子随时挤爆（OOM）。本 PR 让 AR 把杂物收到 70%——他自己依然够用（KV 池仍是需求的 2.35 倍），却给全屋腾出了走道。

可复用的工程思想

- “一行改动”也要拿数据说话。作者给出 A/B 实测，方差为 0、外推说明，把一个看似随手的常数改动论证到无可辩驳。
- 分清“改容量”与“改行为”。明确指出此改动只动 KV 池大小、不动生成数学，输出 bit-identical，从而把评审风险降到最低。
- 过度预留 = 隐性浪费。默认值未必最优，量一量真实需求（131072 token），常能发现 3× 的水分。
- 共享资源系统要为最贪的组件设“预留护栏”（reserve），而不是放任先到先得。

想深入可以再补：KV cache 显存的精确计算（层数 × 头数 × head_dim × 2 × dtype 字节 × token 数）→ 分页 KV cache（PagedAttention）→ GPU 显存分配器与碎片 → 多模型 colocation 的显存调度。

#863 [CI] Calibration 2026/06/22

MERGED

CI

作者 zhaochenyang20 · 合并于 2026-06-22 22:53 UTC · [查看 PR ↗](#)

做了什么：这是一次例行的 CI 性能/质量阈值"再校准"。在 2×H2O 上把 Qwen3-ASR、MOSS-TTS（流式 + 非流式）等工作负载各跑 5 遍，取"5 次里最差的一次（worst-of-5）"作为新的阈值，写回测试文件。

为什么需要：CI 里有一堆"性能不能比基线差太多""WER 不能超过 X%"的硬阈值。但硬件、驱动、随机性会让指标天然波动，阈值设太紧会频繁误报（flaky），太松又拦不住真回退。定期用"最差值"重新标定，能让阈值既贴近真实能力、又留出噪声容忍。

亮点数字：非流式 MOSS 吞吐阈值 `throughput_qps` 从 6.185 收紧到 9.558（**+54.5%**，说明前期 #822/#843 等优化把实际吞吐拉高了，于是阈值也跟着抬高以锁住成果）；流式 WER 阈值从 2.62% 放宽到 3.05%（流式质量波动更大，适当放松避免误报）。

Worst-of-N 标定 / Flaky 测试治理

它是什么：多次重复测量，取最不利的一次作为阈值，而不是取平均。

为什么需要它：阈值要保证"即使运气最差也能过"，否则 CI 会随机红，浪费团队信任。

本 PR 怎么用：每个指标跑 5 次取 worst，再"收紧（优化生效就抬门槛）"或"放宽（波动大就让步）"地写回。

能学到什么

性能是会回退的资产，阈值要随真实能力动态维护。把优化成果用更严的阈值"焊死"，同时对天生抖动的指标留余地——这是性能 CI 长期可信的关键。延伸：统计显著性、回归基线管理、flaky test 检测。

#857 [CI] Restart latest Omni CI for targeted reruns

MERGED

CI

作者 Ratish1 · 合并于 2026-06-22 15:59 UTC · [查看 PR ↗](#)

做了什么：修复 `/tag-and-rerun-ci moss|higgs` 这类斜杠命令的重跑逻辑——之前它会去重跑一个“较旧的失败运行”，而不是最新的那次，导致看起来像是命令只把 CI 取消了。

根因（很有教学价值）：斜杠命令给 PR 打的标签是用 `GITHUB_TOKEN` 加的，而用 `GITHUB_TOKEN` 触发的事件不会再级联触发新的工作流运行（GitHub 防无限循环的安全设定）。所以不能指望“打完标签会自动产生一次新的带标签运行”。旧逻辑又只在“已完成的失败/跳过运行”里挑，于是挑中了过时的那次；当一个更新的运行已存在时，Omni 的 PR 级并发控制还会把这个过时重跑取消掉，UI 上就显示成“命令只取消了 CI”。

怎么修：先按 `(workflow_id, event)` 选出每类工作流的最新一次运行再决定重跑谁；对定向的 moss/higgs 命令，直接对“最新的 PR Omni CI 运行”做整体 full-rerun，让运行时重新读取最新的目标标签；若它还在排队/进行中，则先取消、等取消完成、再整体重跑。还显式安装了 `requests` 因为处理器现在直接调 GitHub REST 接口做取消/重跑。

GITHUB_TOKEN 不级联触发工作流

它是什么：GitHub Actions 的一条安全规则——用自带的

`GITHUB_TOKEN` 做的写操作（如打标签）不会再触发新的工作流，以防自我递归。

为什么需要它：否则一个工作流改仓库 → 触发自己 → 无限循环。

本 PR 怎么用：正因为打标签不会自动产生新运行，所以处理器必须主动用 REST API 去重跑最新那次运行，才能让新标签生效。

能学到什么

自动化里“事件不触发”和“挑错了对象”是最隐蔽的一类 **Bug**。修复关键是：永远显式锁定“最新对象”，并理解平台的事件触发边界（哪些操作不会级联）。延伸：CI/CD 编排、GitHub Actions 并发组（concurrency group）、事件驱动系统的幂等与最新性。

本日报由自动化例程于 2026-06-23 生成 · 数据来源：GitHub [sgl-project/sglang-omni](#) · 抓取窗口 merged $\geq$ 2026-06-22 (过去约 24 小时)

讲解深度优先于覆盖广度：#822 / #843 / #864 为深度解读，#863 / #857 为基础设施简述。